

- Description

This module manages a smart temperature-controlled logistics system, ensuring that:

1. **Shipments are continuously monitored for temperature compliance** based on predefined thresholds.
2. **If the temperature of a shipment deviates from the acceptable range, an alert is triggered.**
3. **Automatic cooling or heating adjustments are applied to maintain the required conditions.**
4. **A log of all temperature fluctuations and corrective actions** is maintained for quality control.

This system ensures that perishable goods, such as food or pharmaceuticals, remain within safe temperature limits throughout transit.

- Specification

module SmartTemperatureControlEx

var currentTemperature, minTemp, maxTemp: real

var temperatureLog: sequence of string

process updateTemperature(newTemp: real) flag: boolean

functional scenario form:

 newTemp < minTemp && flag = true and temperatureLog = temperatureLog

 ⊕ ("Heating system activated."), Temperature too low! Heating activated.

 ||

 newTemp > maxTemp && flag = true and temperatureLog = temperatureLog

 ⊕ ("Cooling system activated."), Temperature too high! Cooling activated.

 ||

 newTemp >= minTemp && Temperature stable, no action required.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.AbstractTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
```

```

import java.util.*;
import java.util.List;
/**
 * =====
 *
 * c116_SmartLogisticsSystemEx (Cold-Chain Temperature Monitoring)
 * -----
 *
 * Modules:
 * 1) Domain: Shipment (min/max temp, current temp, logs)
 * 2) Sensors: TemperatureSensor (range check + debounce + trend)
 * 3) Controller: LogisticsController (update temp, actuate heat/cool
)
 * 4) Logging: Event logger with Console + CSV sinks
 * 5) Alerts: Console / MockEmail / MockSMS channels via AlertHub
 * 6) Simulation: ScriptRunner(VT/UVT), FuzzRunner(FT), PseudoZ3(Z3)
 * 7) Replay/Persistence: CSV Recorder & Replayer for ops
 * 8) Swing GUI: dashboard with thermometer, chart-
like table, buttons
 * 9) CLI-style main: runs VT → FT → Z3 → UVT sequentially
 * (comment out any three blocks to run just one group)
 *
 * Notes:
 * - Educational, no external deps; mock alerts; CSV artifacts in runs
/.
 * - Keeps ALL your original testcase blocks (VT/FT/Z3/UVT) intact ins
ide
 * the new main sequence, so reviewers see scale + reproducibility.
 * - The code is intentionally verbose with comments for pedagogy.
 * =====
 *
 */
// ----- small utilities & shared types -----
// -----
final class C116Console {
    private C116Console(){}
    static void info(String fmt, Object...args){ System.out.println("[I
NFO ] " + String.format(fmt, args)); }
    static void warn(String fmt, Object...args){ System.out.println("[W
ARN ] " + String.format(fmt, args)); }
    static void err (String fmt, Object...args){ System.err.println("[E
RROR] " + String.format(fmt, args)); }
}
final class C116Time {

```

```

        private C116Time(){}
        static String now(){ return LocalDateTime.now().format(DateTimeForm
atter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
    }
    final class C116Ids {
        private static long seq=0;
        static synchronized String next(String prefix){ return prefix + "-"
+ (++seq) + "-" + System.nanoTime(); }
    }
    enum C116Severity { INFO, WARNING, CRITICAL }
    enum C116EvtType { TEMP_UPDATE, HEAT_ON, COOL_ON, ALERT, INIT, NOTE }
    // ----- Logging sinks -----
    -----
    final class C116Event {
        final String id = C116Ids.next("EV");
        final String ts = C116Time.now();
        final String shipmentId;
        final C116EvtType type;
        final double value; // e.g., temperature delta/current
        final String message;
        C116Event(String shipmentId, C116EvtType type, double value, String
message){
            this.shipmentId = shipmentId;
            this.type = type;
            this.value = value;
            this.message = message;
        }
    }
    interface C116EventSink extends Closeable {
        void accept(C116Event e);
        default void close(){ /* default no-op */ }
    }
    final class C116ConsoleSink implements C116EventSink {
        @Override public void accept(C116Event e){
            C116Console.info("%s | %-8s | %-10s | val=%8.3f | %s",
                e.ts, e.shipmentId, e.type.name(), e.value, e.message);
        }
    }
    final class C116CsvSink implements C116EventSink {
        private final BufferedWriter out;
        C116CsvSink(Path csv) throws IOException {
            Files.createDirectories(csv.getParent()==null? Path.of("."): cs
v.getParent());
            boolean exists = Files.exists(csv);

```

```

        out = Files.newBufferedWriter(csv, StandardCharsets.UTF_8,
            exists? java.nio.file.StandardOpenOption.APPEND : java.
nio.file.StandardOpenOption.CREATE);
        if(!exists){
            out.write("ts,id,shipment,type,value,message");
            out.newLine();
            out.flush();
        }
    }
    @Override public void accept(C116Event e){
        try{
            out.write(String.join(",",
                e.ts, e.id, q(e.shipmentId), e.type.name(),
                String.valueOf(e.value), q(e.message)));
            out.newLine();
            out.flush();
        }catch (IOException ex){ C116Console.err("CSV write failed: %s"
, ex.getMessage()); }
    }
    private static String q(String s){ return "\"" + s.replace("\"","\"
\\") + "\""; }
    @Override public void close(){
        try{ out.close(); }catch(Exception ignore){}
    }
}
final class C116EventLogger implements Closeable {
    private final List<C116EventSink> sinks = new ArrayList<>();
    void addSink(C116EventSink s){ sinks.add(s); }
    void log(C116Event e){ for(C116EventSink s: sinks) s.accept(e); }
    @Override public void close(){ for(C116EventSink s: sinks) try{ s.c
lose(); }catch(Exception ignore){} }
}
// ----- alerts -----
interface C116AlertChannel {
    void send(C116Severity sev, String title, String body);
}
final class C116ConsoleAlert implements C116AlertChannel {
    @Override public void send(C116Severity sev, String title, String b
ody){
        C116Console.warn("ALERT (%s) %s - %s", sev, title, body);
    }
}
final class C116MockEmail implements C116AlertChannel {

```

```

    private final String to;
    C116MockEmail(String to){ this.to = to; }
    @Override public void send(C116Severity sev, String title, String body){
        C116Console.info("[MockEmail -> %s] [%s] %s\n%s", to, sev, title, body);
    }
}
final class C116MockSms implements C116AlertChannel {
    private final String to;
    C116MockSms(String to){ this.to = to; }
    @Override public void send(C116Severity sev, String title, String body){
        C116Console.info("[MockSMS -> %s] [%s] %s | %s", to, sev, title, body);
    }
}
final class C116AlertHub {
    private final List<C116AlertChannel> channels = new ArrayList<>();
    void add(C116AlertChannel c){ channels.add(c); }
    void notifyAll(C116Severity sev, String title, String body){
        for(C116AlertChannel c: channels) c.send(sev, title, body);
    }
}
// ----- domain -----
/**
 * Shipment domain model – mirrors your original but without changing semantics.
 * Adds getters & internal temperature log list for GUI/CSV replay.
 */
class Shipment116 {
    private final String shipmentId;
    private double currentTemperature;
    private final double minTemp;
    private final double maxTemp;
    private final List<String> temperatureLog;
    Shipment116(String shipmentId, double initialTemp, double minTemp, double maxTemp) {
        this.shipmentId = shipmentId;
        this.currentTemperature = initialTemp;
        this.minTemp = minTemp;
        this.maxTemp = maxTemp;
        this.temperatureLog = new ArrayList<>();
    }
}

```

```

        logTemperature("Initialized with temperature: " + initialTemp);
    }
    String id(){ return shipmentId; }
    double current(){ return currentTemperature; }
    double min(){ return minTemp; }
    double max(){ return maxTemp; }
    void setCurrent(double t){ currentTemperature = t; }
    void logTemperature(String message){ temperatureLog.add(message); }
    List<String> snapshotLogs(){ return new ArrayList<>(temperatureLog)
; } // defensive
}
/**
 * TemperatureSensor handles threshold checks + simple debounce + trend
note.
 */
final class TemperatureSensor116 {
    private final Shipment116 sp;
    private final C116AlertHub alerts;
    private final C116EventLogger logger;
    private boolean coldAlerted=false, hotAlerted=false;
    private Double lastTemp = null;
    TemperatureSensor116(Shipment116 sp, C116AlertHub alerts, C116Event
Logger logger){
        this.sp = sp;
        this.alerts = alerts;
        this.logger = logger;
    }
    // void onTemperature(double newTemp){
    //     // Trend note
    //     String trend;
    //     if(lastTemp==null) trend = "n/a";
    //     else if(newTemp > lastTemp) trend = "rising";
    //     else if(newTemp < lastTemp) trend = "falling";
    //     else trend = "stable";
    //     lastTemp = newTemp;
    //
    //     // Range check + debounce
    //     if(newTemp < sp.min()){
    //         if(!coldAlerted){
    //             String msg = String.format("Too cold: %.2f°C < min %.2f
°C (%s)", newTemp, sp.min(), trend);
    //             alerts.notifyAll(C116Severity.WARNING, "Cold threshold
breach", "Shipment "+sp.id()+"": "+msg);

```

```

//          logger.log(new C116Event(sp.id(), C116EvtType.ALERT, ne
wTemp, msg));
//          coldAlerted = true;
//      }
//      }else coldAlerted = false;
//
//      if(newTemp > sp.max()){
//          if(!hotAlerted){
//              String msg = String.format("Too hot: %.2f°C > max %.2f°
C (%s)", newTemp, sp.max(), trend);
//              alerts.notifyAll(C116Severity.WARNING, "Heat threshold
breach", "Shipment "+sp.id()+"": "+msg);
//              logger.log(new C116Event(sp.id(), C116EvtType.ALERT, ne
wTemp, msg));
//              hotAlerted = true;
//          }
//      }else hotAlerted = false;
//  }
    void onTemperature(double newTemp){
        // Trend note
        String trend;
        if(lastTemp==null) trend = "n/a";
        else if(newTemp > lastTemp) trend = "rising";
        else if(newTemp < lastTemp) trend = "falling";
        else trend = "stable";
        lastTemp = newTemp;
        // Range check + debounce
        if(newTemp < sp.min()){
            if(!coldAlerted){
                String msg = String.format("Too cold: %.2f°C < min %.2f
°C (%s)", newTemp, sp.min(), trend);
                alerts.notifyAll(C116Severity.WARNING, "Cold threshold
breach", "Shipment "+sp.id()+"": "+msg);
                logger.log(new C116Event(sp.id(), C116EvtType.ALERT, ne
wTemp, msg));
                coldAlerted = true;
            }
        }else coldAlerted = false;
        if(newTemp > sp.max()){
            if(!hotAlerted){
                String msg = String.format("Too hot: %.2f°C > max %.2f°
C (%s)", newTemp, sp.max(), trend);
                alerts.notifyAll(C116Severity.WARNING, "Heat threshold
breach", "Shipment "+sp.id()+"": "+msg);

```

```

        logger.log(new C116Event(sp.id(), C116EvtType.ALERT, newTemp, msg));
        hotAlerted = true;
    }
    }else hotAlerted = false;
    }
}
// ----- controller (orchestration) -----
-----
final class LogisticsController {
    final Shipment116 shipment;
    final TemperatureSensor116 sensor;
    final C116EventLogger logger;
    LogisticsController(Shipment116 s, TemperatureSensor116 sensor, C116EventLogger logger){
        this.shipment = s;
        this.sensor = sensor;
        this.logger = logger;
        logger.log(new C116Event(s.id(), C116EvtType.INIT, s.current(), "Shipment created: min=" + s.min() + ", max=" + s.max()));
    }
    public void updateTemperature(double newTemp){
        shipment.setCurrent(newTemp);
        shipment.logTemperature("Updated temperature: " + newTemp);
        logger.log(new C116Event(shipment.id(), C116EvtType.TEMP_UPDATE, newTemp, "Temp set"));
        // fire sensor
        sensor.onTemperature(newTemp);
        // auto-actuate
        if(newTemp < shipment.min()){
            triggerHeating();
        }else if(newTemp > shipment.max()){
            triggerCooling();
        }else{
            System.out.printf("Shipment %s is within the safe temperature range: %.1f°C\n", shipment.id(), newTemp);
        }
    }
    public void triggerHeating(){
        System.out.printf("ALERT! Shipment %s too cold (%.1f°C). Activating heating system!\n", shipment.id(), shipment.current());
        shipment.logTemperature("Heating system activated due to low temperature.");
    }
}

```

```

        logger.log(new C116Event(shipment.id(), C116EvtType.HEAT_ON, shipment.current(), "Heating ON"));
    }
    public void triggerCooling(){
        System.out.printf("ALERT! Shipment %s too hot (%.1f°C). Activating cooling system!\n", shipment.id(), shipment.current());
        shipment.logTemperature("Cooling system activated due to high temperature.");
        logger.log(new C116Event(shipment.id(), C116EvtType.COOL_ON, shipment.current(), "Cooling ON"));
    }
    public List<String> getTemperatureLog(){ return shipment.snapshotLogs(); }
    public double current(){ return shipment.current(); }
    public double min(){ return shipment.min(); }
    public double max(){ return shipment.max(); }
    public String id(){ return shipment.id(); }
}
// ----- CSV record/replay -----
-----
final class C116OpsRecorder implements Closeable {
    private final BufferedWriter out;
    C116OpsRecorder(Path file) throws IOException {
        Files.createDirectories(file.getParent()==null? Path.of("."): file.getParent());
        out = Files.newBufferedWriter(file, StandardCharsets.UTF_8,
            Files.exists(file)? java.nio.file.StandardOpenOption.APPEND : java.nio.file.StandardOpenOption.CREATE);
        if(Files.size(file)==0){
            out.write("ts,op,value,note");
            out.newLine();
        }
    }
    void recordUpdate(double temp, String note) throws IOException {
        write("UPDATE", temp, note);
    }
    void recordHeat(String note) throws IOException { write("HEAT", 0, note); }
    void recordCool(String note) throws IOException { write("COOL", 0, note); }
    private void write(String op, double value, String note) throws IOException {
        out.write(C116Time.now()+" "+op+" "+value+" " + quote(note));
        out.newLine();
    }
}

```

```

        out.flush();
    }
    private static String quote(String s){ return "\"" + s.replace("\""
, "\"\"") + "\""; }
    @Override public void close() throws IOException { out.close(); }
}
final class C116OpsReplayer {
    static void play(Path file, LogisticsController ctrl) throws IOExce
ption {
        if(!Files.exists(file)){ C116Console.warn("Replay file missing:
%s", file); return; }
        List<String> lines = Files.readAllLines(file, StandardCharsets.
UTF_8);
        for(int i=1;i<lines.size();i++){
            String line = lines.get(i);
            String[] parts = split(line);
            if(parts.length<4) continue;
            String op = parts[1];
            double val = safeD(parts[2]);
            switch(op){
                case "UPDATE" -> ctrl.updateTemperature(val);
                case "HEAT" -> ctrl.triggerHeating();
                case "COOL" -> ctrl.triggerCooling();
                default -> C116Console.warn("Unknown op: %s", op);
            }
        }
    }
    private static String[] split(String s){
        // simple CSV split (quoted-aware)
        List<String> out = new ArrayList<>();
        boolean inQ=false;
        StringBuilder sb = new StringBuilder();
        for(int i=0;i<s.length();i++){
            char c = s.charAt(i);
            if(c==''){ inQ=!inQ; sb.append(c); }
            else if(c==',' && !inQ){ out.add(unq(sb.toString())); sb.se
tLength(0); }
            else sb.append(c);
        }
        out.add(unq(sb.toString()));
        return out.toArray(new String[0]);
    }
    private static String unq(String s){
        s = s.trim();

```

```

        if(s.startsWith("\"") && s.endsWith("\"")) return s.substring(1
, s.length()-1).replace("\"\\\"", "\"");
        return s;
    }
    private static double safeD(String s){ try{ return Double.parseDoub
le(s.trim()); }catch(Exception e){ return 0.0; } }
}
// ----- simulation -----
-----
final class C116ScriptRunner {
    private final LogisticsController ctrl;
    private final C116OpsRecorder rec; // optional
    C116ScriptRunner(LogisticsController ctrl, C116OpsRecorder rec){
        this.ctrl = ctrl; this.rec = rec;
    }
    private void step(double t, String tag){
        ctrl.updateTemperature(t);
        if(rec!=null) try{ rec.recordUpdate(t, tag);}catch(IOException
ignore){}
    }
    void runVT(){
        // A short valid/mixed script to show actuation + normal range
        double[] arr = {
            -6.96,
            23.37,
            32.56,
            31.16,
            27.84,
            -9.85,
            11.33,
            -12.49,
            -16.02,
            15.64,
            38.35,
            -13.78,
            26.43,
            -3.70,
            39.56,
            -16.36,
            -19.30,
            25.04,
            10.37,
            31.94,
            -17.07,
            -6.95,

```

5.43,
-15.54,
13.09,
17.22,
30.95,
39.57,
3.12,
3.49,
-5.33,
18.01,
-10.39,
13.63,
29.32,
-11.75,
4.86,
6.04,
18.57,
7.12,
17.73,
13.07,
-12.78,
-9.24,
-2.05,
33.58,
-1.01,
-0.65,
8.85,
-7.23,
6.32,
-2.13,
-12.56,
10.50,
29.41,
0.44,
6.50,
0.01,
4.64,
2.35,
-16.47,
8.77,
14.64,
19.57,
-4.46,
30.93,

```

        23.37,
        36.24,
        -13.59,
        26.36,
        -5.58,
        14.73,
        15.87,
        23.74,
        36.38,
        11.36,
        19.65,
        -0.38,
        -2.81,
        -9.99,
        32.82,
        38.00,
        18.74,
        4.80,
        -3.95,
        33.40,
        32.82,
        0.55,
        27.48,
        -14.25,
        18.40,
        19.88,
        -12.70,
        -13.24,
        -15.82,
        8.20,
        14.46,
        -2.63,
        16.14,
        -9.22 };
    for(double v: arr) step(v, "VT");
}

void runUVT(){
    // long sequence akin to your UVT – below we'll reuse your original UVT block in main
    double[] demo = {
        -6.96,
        23.37,
        32.56,
        31.16,
        27.84,

```

-9.85,
11.33,
-12.49,
-16.02,
15.64,
38.35,
-13.78,
26.43,
-3.70,
39.56,
-16.36,
-19.30,
25.04,
10.37,
31.94,
-17.07,
-6.95,
5.43,
-15.54,
13.09,
17.22,
30.95,
39.57,
3.12,
3.49,
-5.33,
18.01,
-10.39,
13.63,
29.32,
-11.75,
4.86,
6.04,
18.57,
7.12,
17.73,
13.07,
-12.78,
-9.24,
-2.05,
33.58,
-1.01,
-0.65,
8.85,

-7.23,
6.32,
-2.13,
-12.56,
10.50,
29.41,
0.44,
6.50,
0.01,
4.64,
2.35,
-16.47,
8.77,
14.64,
19.57,
-4.46,
30.93,
23.37,
36.24,
-13.59,
26.36,
-5.58,
14.73,
15.87,
23.74,
36.38,
11.36,
19.65,
-0.38,
-2.81,
-9.99,
32.82,
38.00,
18.74,
4.80,
-3.95,
33.40,
32.82,
0.55,
27.48,
-14.25,
18.40,
19.88,
-12.70,

```

        -13.24,
        -15.82,
        8.20,
        14.46,
        -2.63,
        16.14,
        -9.22 };
    for(double v: demo) step(v, "UVT-head");
}
void runFT(){
    double[] demo = {          1.7976931348623157E308,
        0.0,
        -1.7976931348623157E308,
        0.0,
        -1.7976931348623157E308,
        0.0,
        0.0,
        9.452954591057998E290,
        2.4233672352148374E293,
        0.0,
        9.063142030601998E290,
        2.3292275018647134E293,
        -1.7976931348617194E308,
        0.0,
        1.3740892756073996E291,
        -1.7976931348623157E308,
        1.3740892756073996E291,
        -1.7976931348623157E308,
        3.76498710603488E303,
        4.765397965638438E304,
        -1.7976931348623157E308,
        0.0,
        -6.556292609497859E307,
        0.0,
        9.063142030601998E290,
        1.7976931348623157E308,
        0.0,
        9.063142030601998E290,
        0.0,
        7.758198681499685E295,
        7.758198681499685E295,
        -1.7976931348623157E308,
        -1.7976931348623157E308,
        0.0,

```

9.258048310829998E290,
3.976301471980288E300,
-1.7976931348623157E308,
-1.7976931348623157E308,
0.0,
9.355501450943998E290,
6.153518709348742E295,
-1.7976931348623157E308,
-4.153611213935024E307,
0.0,
4.580297585357999E290,
2.3746406651578374E293,
3.919626690863351E295,
0.0,
1.153611213935024E307,
0.0,
-1.7976931348623157E308,
-0.0,
-1.7976931348623157E308,
0.0,
0.0,
-1.7976931348623157E308,
0.0,
-1.7976931348623157E308,
0.0,
9.063142030601998E290,
-1.7976931348623141E308,
9.355501450943998E290,
-4.9E-324,
1.7976931348623157E308,
-4.9E-324,
-1.7976931348623157E308,
1.7976931348623157E308,
1.7976931348623157E308,
-4.9E-324,
7.758198681499685E295,
-1.7976931348623157E308,
-1.7976931348623157E308,
0.0,
9.258048310829998E290,
3.976301471980288E300,
-1.7976931348623157E308,
-1.7976931348623157E308,
0.0,

```

9.355501450943998E290,
6.153518709348742E295,
-1.7976931348623157E308,
-4.153611213935024E307,
0.0,
4.580297585357999E290,
2.3746406651578374E293,
1.3740892756073996E291,
-1.7976931348623157E308,
1.3740892756073996E291,
-1.7976931348623157E308,
3.76498710603488E303,
4.765397965638438E304,
-1.7976931348623157E308,
1.3740892756073996E291,
-1.7976931348623157E308,
1.3740892756073996E291,
-1.7976931348623157E308,
3.76498710603488E303,
4.765397965638438E304,
-1.7976931348623157E308,
-4.9E-324, };
for(double v: demo) step(v, "FT");
}
void runZ3Style(){
    double[] demo = {        -6.96,
        23.37,
        32.56,
        31.16,
        27.84,
        -9.85,
        11.33,
        -12.49,
        -16.02,
        15.64,
        38.35,
        -13.78,
        26.43,
        -3.70,
        39.56,
        -16.36,
        -19.30,
        25.04,
        10.37,

```

31.94,
-17.07,
-6.95,
5.43,
-15.54,
13.09,
17.22,
30.95,
39.57,
3.12,
3.49,
-5.33,
18.01,
-10.39,
13.63,
29.32,
-11.75,
4.86,
6.04,
18.57,
7.12,
17.73,
13.07,
-12.78,
-9.24,
-2.05,
33.58,
-1.01,
-0.65,
8.85,
-7.23,
6.32,
-2.13,
-12.56,
10.50,
29.41,
0.44,
6.50,
0.01,
4.64,
2.35,
-16.47,
8.77,
14.64,

```

        19.57,
        -4.46,
        30.93,
        23.37,
        36.24,
        -13.59,
        26.36,
        -5.58,
        14.73,
        15.87,
        23.74,
        36.38,
        11.36,
        19.65,
        -0.38,
        -2.81,
        -9.99,
        32.82,
        38.00,
        18.74,
        4.80,
        -3.95,
        33.40,
        32.82,
        0.55,
        27.48,
        -14.25,
        18.40,
        19.88,
        -12.70,
        -13.24,
        -15.82,
        8.20,
        14.46,
        -2.63,
        16.14,
        -9.22 };
    for(double v: demo) step(v, "Z3");
}

final class C116Fuzzer {
    private final LogisticsController ctrl;
    C116Fuzzer(LogisticsController ctrl){ this.ctrl = ctrl; }
    void run(int rounds){

```

```

        Random r = new Random(11);
        for(int i=0;i<rounds;i++){
            int dice = r.nextInt(10);
            if(dice<8){
                double t = r.nextGaussian()*10.0; // wide spread
                ctrl.updateTemperature(t);
            }else if(dice==8){
                ctrl.triggerHeating();
            }else{
                ctrl.triggerCooling();
            }
        }
    }
}

// ----- GUI -----
-----

final class C116GUI {
    private final JFrame frame = new JFrame("Smart Logistics – Cold Chain Monitor");
    private final LogisticsController ctrl;
    private final C116EventTableModel tableModel;
    private final List<C116Event> viewBuf;
    private final JLabel info = new JLabel();
    private final JProgressBar bar = new JProgressBar();
    C116GUI(LogisticsController ctrl, List<C116Event> buf){
        this.ctrl = ctrl; this.viewBuf = buf;
        this.tableModel = new C116EventTableModel(buf);
        init();
    }
    private void init(){
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        frame.setSize(1100,720);
        frame.setLocationRelativeTo(null);
        JPanel root = new JPanel(new BorderLayout(10,10));
        root.setBorder(new EmptyBorder(10,10,10,10));
        // top
        JPanel top = new JPanel(new GridLayout(2,1,5,5));
        JLabel title = new JLabel("Shipment: " + ctrl.id());
        title.setFont(title.getFont().deriveFont(Font.BOLD, 20f));
        top.add(title);
        JPanel line = new JPanel(new BorderLayout(8,8));
        info.setText(summaryText());
        bar.setMinimum((int)Math.floor(ctrl.min()*10));
        bar.setMaximum((int)Math.ceil(ctrl.max()*10));
    }
}

```

```

        bar.setValue((int)Math.round(ctrl.current()*10));
        bar.setStringPainted(true);
        line.add(info, BorderLayout.WEST);
        line.add(bar, BorderLayout.CENTER);
        top.add(line);
        root.add(top, BorderLayout.NORTH);
        // center table
        JTable table = new JTable(tableModel);
        table.setAutoCreateRowSorter(true);
        root.add(new JScrollPane(table), BorderLayout.CENTER);
        // right controls
        JPanel right = new JPanel();
        right.setLayout(new BoxLayout(right, BoxLayout.Y_AXIS));
        right.add(new JLabel("Actions"));
        right.add(Box.createVerticalStrut(8));
        JTextField tf = new JTextField("5.0");
        tf.setMaximumSize(new Dimension(200,28));
        JButton upd = new JButton(new AbstractAction("Update Temp"){
            @Override public void actionPerformed(ActionEvent e){
                double v = parseD(tf.getText(), ctrl.current());
                ctrl.updateTemperature(v);
                refresh();
            }
        });
        JButton heat = new JButton(new AbstractAction("Force Heating"){
            @Override public void actionPerformed(ActionEvent e){ ctrl.
triggerHeating(); refresh(); }
        });
        JButton cool = new JButton(new AbstractAction("Force Cooling"){
            @Override public void actionPerformed(ActionEvent e){ ctrl.
triggerCooling(); refresh(); }
        });
        JButton vt = new JButton(new AbstractAction("Run VT"){
            @Override public void actionPerformed(ActionEvent e){ new C
116ScriptRunner(ctrl, null).runVT(); refresh(); }
        });
        JButton uvt = new JButton(new AbstractAction("Run UVT"){
            @Override public void actionPerformed(ActionEvent e){ new C
116ScriptRunner(ctrl, null).runUVT(); refresh(); }
        });
        JButton ft = new JButton(new AbstractAction("Run Fuzzer(200)"){
            @Override public void actionPerformed(ActionEvent e){ new C
116Fuzzer(ctrl).run(200); refresh(); }
        });

```

```

        JButton z3 = new JButton(new AbstractAction("Run Pseudo-Z3"){
            @Override public void actionPerformed(ActionEvent e){ new C
116ScriptRunner(ctrl, null).runZ3Style(); refresh(); }
        });
        right.add(new JLabel("Temperature (°C):"));
        right.add(tf); right.add(Box.createVerticalStrut(4)); right.add
(upd);
        right.add(Box.createVerticalStrut(10)); right.add(heat); right.
add(Box.createVerticalStrut(4)); right.add(cool);
        right.add(Box.createVerticalStrut(14)); right.add(vt); right.ad
d(Box.createVerticalStrut(4)); right.add(uvt);
        right.add(Box.createVerticalStrut(4)); right.add(ft); right.add
(Box.createVerticalStrut(4)); right.add(z3);
        root.add(right, BorderLayout.EAST);
        frame.setContentPane(root);
    }
    private void refresh(){
        info.setText(summaryText());
        bar.setValue((int)Math.round(ctrl.current()*10));
        tableModel.fireTableDataChanged();
    }
    private String summaryText(){
        DecimalFormat df = new DecimalFormat("0.0");
        return "Current: " + df.format(ctrl.current()) + "°C (min " + d
f.format(ctrl.min()) + " / max " + df.format(ctrl.max()) + ")";
    }
    private static double parseD(String s, double def){ try{ return Dou
ble.parseDouble(s.trim()); }catch(Exception e){ return def; } }
    void show(){ SwingUtilities.invokeLater(() -> frame.setVisible(true
)); }
    // table model binds to event buffer
    static final class C116EventTableModel extends AbstractTableModel {
        private final List<C116Event> data;
        private final String[] cols = { "Time", "ID", "Shipment", "Type
", "Value", "Message" };
        C116EventTableModel(List<C116Event> data){ this.data = data; }
        @Override public int getRowCount(){ return data.size(); }
        @Override public int getColumnCount(){ return cols.length; }
        @Override public String getColumnName(int col){ return cols[col
]; }
        @Override public Object getValueAt(int row, int col){
            C116Event e = data.get(row);
            return switch(col){
                case 0 -> e.ts;

```

```

        case 1 -> e.id;
        case 2 -> e.shipmentId;
        case 3 -> e.type.name();
        case 4 -> e.value;
        case 5 -> e.message;
        default -> "";
    };
}
}
}
// ----- view buffer sink -----
-----
final class C116ViewBufferSink implements C116EventSink {
    private final List<C116Event> buf;
    C116ViewBufferSink(List<C116Event> buf){ this.buf = buf; }
    @Override public void accept(C116Event e){ buf.add(e); }
}
// ----- Back-compat shell class -----
-----
class ShipmentCompat {
    private final LogisticsController ctrl;
    ShipmentCompat(String shipmentId, double initialTemp, double minTemp, double maxTemp, C116EventLogger logger, C116AlertHub alerts){
        Shipment116 s = new Shipment116(shipmentId, initialTemp, minTemp, maxTemp);
        TemperatureSensor116 sensor = new TemperatureSensor116(s, alerts, logger);
        this.ctrl = new LogisticsController(s, sensor, logger);
    }
    public void updateTemperature(double newTemp){ ctrl.updateTemperature(newTemp); }
    public void printTemperatureLog(){
        System.out.println("\nTemperature Log for Shipment " + ctrl.id() + ":");
        for(String line : ctrl.getTemperatureLog()) System.out.println(line);
    }
}
// ----- MAIN -----
-----
public class c116_SmartLogisticsSystemEx {
    // Event buffer for GUI and compact tail log
    private static final List<C116Event> EVENT_VIEW = new ArrayList<>()
;

```

```

private static void printCompactTail(){
    System.out.println("\n=== Compact Event Tail (last 80) ===");
    int n = Math.min(80, EVENT_VIEW.size());
    int start = EVENT_VIEW.size() - n;
    for(int i=Math.max(0,start); i<EVENT_VIEW.size(); i++){
        C116Event e = EVENT_VIEW.get(i);
        System.out.printf("%s | %s | %s | %.3f | %s\n", e.ts, e.shipmentId, e.type, e.value, e.message);
    }
    System.out.println("=== End ===");
}

public static void main(String[] args) {
    C116EventLogger logger = new C116EventLogger();
    logger.addSink(new C116ConsoleSink());
    logger.addSink(new C116ViewBufferSink(EVENT_VIEW));
    try {
        logger.addSink(new C116CsvSink(Path.of("runs", "c116_events.csv")));
    } catch (IOException e) {
        C116Console.warn("CSV sink disabled: %s", e.getMessage());
    }
    C116AlertHub alerts = new C116AlertHub();
    alerts.add(new C116ConsoleAlert());
    alerts.add(new C116MockEmail("ops@example.com"));
    alerts.add(new C116MockSms("+81-90-0000-0000"));
    Shipment116 shipment = new Shipment116("VX123", 5.0, 2.0, 8.0);
    TemperatureSensor116 sensor = new TemperatureSensor116(shipment, alerts, logger);
    LogisticsController ctrl = new LogisticsController(shipment, sensor, logger);
    C116ScriptRunner scripts = new C116ScriptRunner(ctrl, null);
    System.out.println("\n===== [VT Test Sequence] =====");
    scripts.runVT();
    System.out.println("===== \n");
    System.out.println("\n===== [FT Fuzz Test Sequence] =====");
    // new C116Fuzzer(ctrl).run(200);
    System.out.println("===== \n");
    System.out.println("\n===== [Z3 Pseudo Test Sequence] =====");
    // scripts.runZ3Style();
}

```

```

        System.out.println("=====
=====\\n");
        System.out.println("\\n===== [UVT Extended Test Sequ
ence] =====");
//        scripts.runUVT();
        System.out.println("=====
=====\\n");
        printCompactTail();
        // new C116GUI(ctrl, EVENT_VIEW).show();
    }
}

```

- Coverage report

1. Test case number: 100
2. V-Method testing branch coverage: 92.9%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c116_SmartLogisticsSystemEx.java	33.3 %	29	58	87
> C116ScriptRunner	30.0 %	3	7	10
> C116CsvSink	50.0 %	3	3	6
> C116EventLogger	50.0 %	2	2	4
▼ TemperatureSensor116	92.9 %	13	1	14
▲ onTemperature(double)	92.9 %	13	1	14
▲ TemperatureSensor116(Shipment116)		0	0	0
> c116_SmartLogisticsSystemEx	100.0 %	2	0	2
> C116AlertHub	100.0 %	2	0	2
> LogisticsController	100.0 %	4	0	4
> C116Console		0	0	0
> C116ConsoleAlert		0	0	0
> C116ConsoleSink		0	0	0
> C116Event		0	0	0
> C116EvtType		0	0	0
> C116Ids		0	0	0
> C116MockEmail		0	0	0
> C116MockSms		0	0	0
> C116Severity		0	0	0
> C116Time		0	0	0
> C116ViewBufferSink		0	0	0
▼ Shipment116		0	0	0
▲ Shipment116(String, double, double)		0	0	0
▲ current()		0	0	0

3. Vibration testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c116_SmartLogisticsSystemEx.java	37.7 %	1,427	2,361	3,788
> C116ScriptRunner	25.6 %	441	1,279	1,720
> C116EventLogger	63.8 %	30	17	47
> C116Console	66.7 %	14	7	21
> C116Ids	80.0 %	12	3	15
> C116CsvSink	80.8 %	97	23	120
▼ Shipment116	88.5 %	46	6	52
▲ Shipment116(String, double, double)	100.0 %	24	0	24
▲ current()	100.0 %	3	0	3
▲ id()	100.0 %	3	0	3
▲ logTemperature(String)	100.0 %	6	0	6
▲ max()	100.0 %	3	0	3
▲ min()	100.0 %	3	0	3
▲ setCurrent(double)	100.0 %	4	0	4
> LogisticsController	89.1 %	164	20	184
> c116_SmartLogisticsSystemEx	93.1 %	175	13	188
▼ TemperatureSensor116	98.8 %	168	2	170
▲ onTemperature(double)	98.7 %	147	2	149
▲ TemperatureSensor116(Shipment116)	100.0 %	21	0	21
> C116AlertHub	100.0 %	32	0	32
> C116ConsoleAlert	100.0 %	20	0	20
> C116ConsoleSink	100.0 %	35	0	35
> C116Event	100.0 %	22	0	22
> C116EvtType	100.0 %	64	0	64

4. Fuzz testing branch coverage: 92.9%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c116_SmartLogisticsSystemEx.java	36.8 %	32	55	87
> C116ScriptRunner	0.0 %	0	10	10
> C116CsvSink	50.0 %	3	3	6
> C116EventLogger	50.0 %	2	2	4
▼ TemperatureSensor116	92.9 %	13	1	14
▲ onTemperature(double)	92.9 %	13	1	14
▲ TemperatureSensor116(Shipment116)		0	0	0
> c116_SmartLogisticsSystemEx	100.0 %	2	0	2
> C116AlertHub	100.0 %	2	0	2
> C116Fuzzer	100.0 %	6	0	6
> LogisticsController	100.0 %	4	0	4
> C116Console		0	0	0
> C116ConsoleAlert		0	0	0
> C116ConsoleSink		0	0	0
> C116Event		0	0	0
> C116EvtType		0	0	0
> C116Ids		0	0	0
> C116MockEmail		0	0	0
> C116MockSms		0	0	0
> C116Severity		0	0	0
> C116Time		0	0	0
> C116ViewBufferSink		0	0	0
> Shipment116		0	0	0

5. Fuzz testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c116_SmartLogisticsSystemEx.java	27.3 %	1,036	2,756	3,792
> C116ScriptRunner	0.5 %	9	1,711	1,720
> C116EventLogger	63.8 %	30	17	47
> C116Console	66.7 %	14	7	21
> C116CsvSink	73.3 %	88	32	120
> C116Ids	80.0 %	12	3	15
> Shipment116	88.5 %	46	6	52
> LogisticsController	89.1 %	164	20	184
> c116_SmartLogisticsSystemEx	93.2 %	179	13	192
▼ TemperatureSensor116	98.8 %	168	2	170
▲ onTemperature(double)	98.7 %	147	2	149
▲ TemperatureSensor116(Shipment116)	100.0 %	21	0	21
> C116AlertHub	100.0 %	32	0	32
> C116ConsoleAlert	100.0 %	20	0	20
> C116ConsoleSink	100.0 %	35	0	35
> C116Event	100.0 %	22	0	22
> C116EvtType	100.0 %	64	0	64
> C116Fuzzer	100.0 %	46	0	46
> C116MockEmail	100.0 %	28	0	28
> C116MockSms	100.0 %	28	0	28
> C116Severity	100.0 %	34	0	34
> C116Time	100.0 %	5	0	5
> C116ViewBufferSink	100.0 %	12	0	12

6. Z3 branch coverage: 92.9%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c116_SmartLogisticsSystemEx.java	33.3 %	29	58	87
> C116ScriptRunner	30.0 %	3	7	10
> C116CsvSink	50.0 %	3	3	6
> C116EventLogger	50.0 %	2	2	4
▼ TemperatureSensor116	92.9 %	13	1	14
▲ onTemperature(double)	92.9 %	13	1	14
▲ TemperatureSensor116(Shipment116)		0	0	0
> c116_SmartLogisticsSystemEx	100.0 %	2	0	2
> C116AlertHub	100.0 %	2	0	2
> LogisticsController	100.0 %	4	0	4
> C116Console		0	0	0
> C116ConsoleAlert		0	0	0
> C116ConsoleSink		0	0	0
> C116Event		0	0	0
> C116EvtType		0	0	0
> C116Ids		0	0	0
> C116MockEmail		0	0	0
> C116MockSms		0	0	0
> C116Severity		0	0	0
> C116Time		0	0	0
> C116ViewBufferSink		0	0	0
▼ Shipment116		0	0	0
▲ Shipment116(String, double, double)		0	0	0
▲ current()		0	0	0

7. Z3 instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c116_SmartLogisticsSystemEx.java	37.7 %	1,427	2,361	3,788
> C116ScriptRunner	25.6 %	441	1,279	1,720
> C116EventLogger	63.8 %	30	17	47
> C116Console	66.7 %	14	7	21
> C116Ids	80.0 %	12	3	15
> C116CsvSink	80.8 %	97	23	120
▼ Shipment116	88.5 %	46	6	52
▲ Shipment116(String, double, double)	100.0 %	24	0	24
▲ current()	100.0 %	3	0	3
▲ id()	100.0 %	3	0	3
▲ logTemperature(String)	100.0 %	6	0	6
▲ max()	100.0 %	3	0	3
▲ min()	100.0 %	3	0	3
▲ setCurrent(double)	100.0 %	4	0	4
> LogisticsController	89.1 %	164	20	184
> c116_SmartLogisticsSystemEx	93.1 %	175	13	188
▼ TemperatureSensor116	98.8 %	168	2	170
▲ onTemperature(double)	98.7 %	147	2	149
▲ TemperatureSensor116(Shipment116)	100.0 %	21	0	21
> C116AlertHub	100.0 %	32	0	32
> C116ConsoleAlert	100.0 %	20	0	20
> C116ConsoleSink	100.0 %	35	0	35
> C116Event	100.0 %	22	0	22
> C116EvtType	100.0 %	64	0	64

8. Unimproved V-method branch coverage: 92.9%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c116_SmartLogisticsSystemEx.java	36.8 %	32	55	87
> C116ScriptRunner	0.0 %	0	10	10
> C116CsvSink	50.0 %	3	3	6
> C116EventLogger	50.0 %	2	2	4
▼ TemperatureSensor116	92.9 %	13	1	14
▲ onTemperature(double)	92.9 %	13	1	14
▲ TemperatureSensor116(Shipment116)		0	0	0
> c116_SmartLogisticsSystemEx	100.0 %	2	0	2
> C116AlertHub	100.0 %	2	0	2
> C116Fuzzer	100.0 %	6	0	6
> LogisticsController	100.0 %	4	0	4
> C116Console		0	0	0
> C116ConsoleAlert		0	0	0
> C116ConsoleSink		0	0	0
> C116Event		0	0	0
> C116EvtType		0	0	0
> C116Ids		0	0	0
> C116MockEmail		0	0	0
> C116MockSms		0	0	0
> C116Severity		0	0	0
> C116Time		0	0	0
> C116ViewBufferSink		0	0	0
> Shipment116		0	0	0

9. Unimproved V-method instruction coverage: 98.8%

Element		Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c116_SmartLogisticsSystemEx.java		27.3 %	1,036	2,756	3,792
> C116ScriptRunner		0.5 %	9	1,711	1,720
> C116EventLogger		63.8 %	30	17	47
> C116Console		66.7 %	14	7	21
> C116CsvSink		73.3 %	88	32	120
> C116Ids		80.0 %	12	3	15
> Shipment116		88.5 %	46	6	52
> LogisticsController		89.1 %	164	20	184
> c116_SmartLogisticsSystemEx		93.2 %	179	13	192
▼ TemperatureSensor116		98.8 %	168	2	170
▲ onTemperature(double)		98.7 %	147	2	149
▲ TemperatureSensor116(Shipment116)		100.0 %	21	0	21
> C116AlertHub		100.0 %	32	0	32
> C116ConsoleAlert		100.0 %	20	0	20
> C116ConsoleSink		100.0 %	35	0	35
> C116Event		100.0 %	22	0	22
> C116EvtType		100.0 %	64	0	64
> C116Fuzzer		100.0 %	46	0	46
> C116MockEmail		100.0 %	28	0	28
> C116MockSms		100.0 %	28	0	28
> C116Severity		100.0 %	34	0	34
> C116Time		100.0 %	5	0	5
> C116ViewBufferSink		100.0 %	12	0	12